

OCS-vs-PS for LLM training networks

findings, the experiment loop, and what's next

To: Ahmed

From: Sahas

Date: 2026-06-14

Ahmed —

Quick brain-dump on where the optical-circuit-switch (OCS) vs packet-switch (PS) study stands, how the simulation loop is wired now, and the experiments I think are worth running next.

TL;DR

For an 8B model on 400 Gbps-class links, the network is a **few-percent effect whichever way you cut it**. The OCS advantage decomposes into two dimensions and we can now measure each cleanly:

- **Algorithm** (direct all-reduce on dedicated circuits vs ring): **~0.3%** at 50 GB/s.
- **Congestion** (no fabric contention vs a thin shared PS spine): **~3.8%**.

The congestion dimension is $\sim 10\times$ the algorithm dimension — so the OCS value is real and it lives where we expected (PP traffic on the shared spine) — but both are small *for this workload* because it's bubble/compute-bound. The interesting next question is the regime where comm gets exposed on the critical path and these numbers grow.

Findings so far

1. Synthetic sweeps (8B / 70B / 405B at 64–512 ranks):

- OCS sensitivity is dominated by **DP degree, not model size**. 8B/70B at DP=8 need circuit count $C \geq 128$ (current Palomar port count) to stay within $\sim 1\%$ of PS; 405B at DP=2 is essentially OCS-immune at any realistic C . The dangerous scaling path for fabric design is growing ranks by **doubling DP, not PP**.
- **Circuit setup latency** (T_{setup}) is **irrelevant** at commercial MEMS timescales (≤ 27 ms). 1F1B pipeline scheduling leaves 100 ms–5 s of natural slack between a PP send completing and its data being consumed, which absorbs the switching time entirely. **Capacity (C), not latency, is the binding constraint**.

2. The 16-GPU OCS-vs-PS deep dive (8B, TP1/PP2/DP8) — this is the controlled experiment:

- Built a coupled forward DAG scheduler for the OCS side and validated it to **~0.3% against AstraSim** on two independent gates (infinite-bandwidth reproduces the ideal floor; ring @ 50 GB/s reproduces our earlier coupled numbers). So the OCS number is artifact-free and trustworthy.
- Against a thin 8:1-oversubscribed Clos PS baseline (packet-level ns-3, HPCC): **stage-0 step +3.8%, stage-1 +4.0%** over the OCS floor.
- The per-flow breakdown is the thesis in one table: **PP cross-stage sends pay $5.6\times$ congestion** (worst flow $8.7\times$) on the thin shared spine, while **DP all-reduce stays intra-pod and is comparable or even better** on the Clos. The contention is on the spine, and it's PP traffic that pays — exactly what OCS's dedicated circuits erase. The reason it's “only” +3.8% at the step level is that a $5.6\times$ penalty on PP sends is ~ 183 ms, and most of that comm hides behind compute.

The experiment loop we’ve set up

The whole pipeline runs on CPU — no GPU cluster needed for the analysis:

1. **Workload generation — STAGE** (symbolic tensor graph) synthesizes Chakra execution traces for distributed LLM workloads symbolically. Validated at 0.23% comm-volume error vs a real 128-GPU H100 run. One trace freezes the workload; network/system are separate config, so a trace is reusable across every network sweep.
2. **OCS side — three engines, each answering a different question:**
 - `ocs_replay` — delay propagator for capacity- C contention sweeps (“how much does a C -circuit fabric stretch the step”).
 - `dag_sim` — per-rank roofline + bandwidth-optimal direct collective cost model.
 - `coupled_sim` — the newest one: a global forward DAG scheduler that combines `dag_sim`’s direct cost model with cross-rank coupling (PP send→recv edges, collective barriers). It’s the only engine that can model a *faster algorithm* and the PP bubble at the same time. The bubble emerges from the graph rather than being hacked in.
3. **PS ground truth — ns-3** packet-level simulation on real fat-tree / thin-Clos topologies with HPCC congestion control.
4. **Common anchor — an ideal floor** (infinite bandwidth) so OCS and PS are each reported as *overhead above* the unavoidable compute+latency floor, which lets us compare the analytical OCS backend and the packet-level PS backend apples-to-apples.

A 16-rank ns-3 run is ~40 min wall; the analytical/coupled engines are seconds.

Potential experiments

A. Push into the OCS-favorable regime (highest value). The +3.8% is gated by comm-hiding, not the size of the congestion. The knobs that expose comm on the critical path and should grow the gap:

- **Lower link bandwidth** (100–200 Gbps) — we already see direct–ring go 0.3% → 4.1% as bandwidth drops 50 → 5 GB/s.
- **Larger DP degree** (the synthetic sweeps say this dominates).
- **Smaller compute per step / deeper pipelines with tighter bubbles.**

B. Scale the controlled experiment to 70B / 405B. Right now the big models only have synthetic-sweep numbers; the coupled-vs-ns-3 deep dive is 8B only.

C. A controlled 4:1-vs-8:1 spine A/B. The current thin-Clos result mixes two variables (we had to drop to 8:1 / 400 Gbps because this ns-3 HPCC build rejects non-standard link rates). A clean single-variable oversubscription sweep would isolate the spine effect.

D. A QoS-aware PS baseline. Our current PS is best-effort (no priority queuing), which is the *conservative* comparison for OCS. Worth documenting what QoS would neutralize.

E. Real trace capture as validation (optional). Phases 0–1 are designed: pure-DP and TP/PP/DP-hybrid 8B on a single 8-GPU node, ~\$25–75. Validates STAGE’s synthetic traces against real op structure.

Happy to walk through any of this or share the lab notes. My instinct is **A** is where the story gets compelling — everything so far says the 8B / 400 Gbps point is just too easy on the network for OCS to shine, and the honest framing is “here’s the workload regime where circuits start to matter.”

